

✓ Data Preprocessing

```
import numpy as np
import pandas as pd
import os
from pathlib import Path
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')

BASE_DIR = Path('/content/drive/MyDrive/AI-ML-Health-Project/data')
JH_SEG_DIR = BASE_DIR / 'JH_seg_labels'
METADATA_PATH = BASE_DIR / 'IARCIImageBankVIA' / 'Cases Meta data JH.xls'

VIA_MAPPING = {
    'Negative': 0,
    'Suspicious of cancer': 1,
    'Positive': 2,
    'Suspicious': 1
}

def load_metadata(metadata_path):
    df = pd.read_excel(metadata_path)
    df['VIA'] = df['VIA'].astype(str).str.strip()
    df['CaseNumber'] = df['CaseNumber'].astype(str).str.strip()
    df['CaseID'] = df['CaseID'].astype(str).str.strip()
    df['Label'] = df['VIA'].map(VIA_MAPPING)
    return df

metadata_df = load_metadata(METADATA_PATH)

def load_jh_data(jh_dir, metadata_df):
    image_paths = []
    mask_paths = []
    classification_labels = []
    case_ids = []
    folder_names = []

    caseid_to_label = {}
    jh_caseid_to_label = {}
```

```

- - -
for _, row in metadata_df.iterrows():
    case_id = str(row['CaseID']).strip()
    label = row['Label']

    if pd.notna(label) and case_id and case_id != 'nan':
        caseid_to_label[case_id] = label
        if case_id.upper().startswith('JH'):
            jh_caseid_to_label[case_id.upper()] = label

jh_path = Path(jh_dir)
case_folders = [f for f in jh_path.iterdir() if f.is_dir()]

for folder in case_folders:
    folder_name = folder.name.strip()
    img_file = folder / 'img.png'
    label_file = folder / 'label.png'
    label = None
    matched_case_id = None

    folder_name_upper = folder_name.upper()
    if folder_name_upper in caseid_to_label:
        label = caseid_to_label[folder_name_upper]
        matched_case_id = folder_name_upper
    elif folder_name_upper in jh_caseid_to_label:
        label = jh_caseid_to_label[folder_name_upper]
        matched_case_id = folder_name_upper

    if label is None and folder_name.startswith('JH'):
        for case_id, lbl in jh_caseid_to_label.items():
            if case_id == folder_name_upper:
                label = lbl
                matched_case_id = case_id
                break

    if label is None and folder_name.startswith('JH'):
        for case_id, lbl in jh_caseid_to_label.items():
            if folder_name_upper in case_id or case_id in folder_name:
                label = lbl
                matched_case_id = case_id
                break

    image_paths.append(str(img_file))
    mask_paths.append(str(label_file))
    classification_labels.append(label)

```

```

        case_ids.append(matched_case_id if matched_case_id else 'Unknown')
        folder_names.append(folder_name)

    return (np.array(image_paths), np.array(mask_paths), np.array(class_labels))

images, masks, labels, case_ids, folder_names = load_jh_data(JH_SEG_DIR)

# Drive already mounted at /content/drive; to attempt to forcibly remount

```

```

import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.utils import class_weight
import numpy as np
import os

physical_devices = tf.config.list_physical_devices('GPU')
if len(physical_devices) > 0:
    tf.config.experimental.set_memory_growth(physical_devices[0], True)

IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32
MODEL_PATH = '/content/drive/MyDrive/AI-ML-Health-Project/densenet_best_model.h5'

X_paths = np.array(images)
y_labels = np.array(labels)
m_paths = np.array(masks)

from collections import Counter

indices = np.arange(len(X_paths))
try:
    train_idx, temp_idx = train_test_split(indices, test_size=0.2, random_state=42)
except ValueError as e:
    train_idx, temp_idx = train_test_split(indices, test_size=0.2, random_state=42)

temp_labels = y_labels[temp_idx]
temp_counts = Counter(temp_labels)

# val, test split
if min(temp_counts.values()) < 2:
    val_idx, test_idx = train_test_split(temp_idx, test_size=0.5, random_state=42)
else:
    val_idx, test_idx = train_test_split(temp_idx, test_size=0.5, random_state=42)

```

```

train_paths = X_paths[train_idx]
train_labels = y_labels[train_idx]
train_masks = m_paths[train_idx]

val_paths = X_paths[val_idx]
val_labels = y_labels[val_idx]
val_masks = m_paths[val_idx]

test_paths = X_paths[test_idx]
test_labels = y_labels[test_idx]
test_masks = m_paths[test_idx]

class_weights_vals = class_weight.compute_class_weight(class_weight='l
class_weights_dict = dict(enumerate(class_weights_vals))

def process_classification_data(image_path, label, is_training=True):
    img = tf.io.read_file(image_path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.resize(img, (224, 224))

    # Data Augmentation
    if is_training:
        img = tf.image.random_flip_left_right(img)
        img = tf.image.random_brightness(img, max_delta=0.2)
        img = tf.image.random_contrast(img, lower=0.8, upper=1.2)
        img = tf.image.rot90(img, k=tf.random.uniform(shape=[], minva

    label = tf.one_hot(label, depth=3)
    return img, label

def create_dataset(paths, labels, is_training=True):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if is_training: ds = ds.shuffle(buffer_size=1000)
    ds = ds.map(lambda x, y: process_classification_data(x, y, is_tra
    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

train_ds = create_dataset(train_paths, train_labels, is_training=True)
val_ds = create_dataset(val_paths, val_labels, is_training=False)
test_ds = create_dataset(test_paths, test_labels, is_training=False)

```

```

from tensorflow.keras import layers, models, backend as K

```

```

def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + smooth)

@tf.keras.utils.register_keras_serializable()
def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

def mean_iou(y_true, y_pred):
    y_pred = tf.argmax(y_pred, axis=-1)
    y_true = tf.argmax(y_true, axis=-1)
    return tf.reduce_mean(tf.keras.metrics.mean_iou(y_true, y_pred, 3))

@tf.keras.utils.register_keras_serializable()
def iou_metric(y_true, y_pred, smooth=1e-6):
    y_pred_f = tf.cast(y_pred > 0.5, tf.float32)
    y_true_f = tf.cast(y_true, tf.float32)

    y_true_f = tf.reshape(y_true_f, [-1])
    y_pred_f = tf.reshape(y_pred_f, [-1])

    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    union = tf.reduce_sum(y_true_f) + tf.reduce_sum(y_pred_f) - intersection
    return (intersection + smooth) / (union + smooth)

def process_segmentation_data(image_path, mask_path, is_training=True):
    img = tf.io.read_file(image_path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.resize(img, (224, 224))

    mask = tf.io.read_file(mask_path)
    mask = tf.image.decode_png(mask, channels=3)
    mask = tf.image.resize(mask, (224, 224), method='nearest')

    mask = tf.cast(mask, tf.float32)

    if is_training:
        combined = tf.concat([img, mask], axis=-1)
        combined = tf.image.random_flip_left_right(combined)
        combined = tf.image.random_flip_up_down(combined)
        img, mask = tf.split(combined, [3, 3], axis=-1)

```

```

img = tf.image.random_brightness(img, max_delta=0.2)
img = tf.image.random_contrast(img, lower=0.8, upper=1.2)

img = tf.cast(img, tf.float32)
mask = mask[..., 0]
mask = tf.cast(mask > 100, tf.int32)
mask = tf.one_hot(mask, depth=2)
return img, mask

def create_seg_dataset(paths, masks, is_training=True):
    ds = tf.data.Dataset.from_tensor_slices((paths, masks))
    if is_training: ds = ds.shuffle(buffer_size=1000)
    ds = ds.map(lambda x, y: process_binary_seg(x, y, is_training=is_training))
    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

def build_unet_binary(classifier_path):
    full_classifier = tf.keras.models.load_model(classifier_path)
    backbone = full_classifier.layers[0]

    layers_to_extract = ['block2b_add', 'block3b_add', 'block5c_add',
                        'encoder_outputs']
    enc_outputs = [backbone.get_layer(name).output for name in layers_to_extract]
    encoder = models.Model(inputs=backbone.input, outputs=enc_outputs)
    s1, s2, s3, bridge = encoder(backbone.input)

    x = layers.Conv2DTranspose(256, 3, strides=2, padding='same')(bridge)
    x = layers.Concatenate()([x, s3])
    x = layers.Conv2D(256, 3, activation='relu', padding='same')(x)

    x = layers.Conv2DTranspose(128, 3, strides=2, padding='same')(x)
    x = layers.Concatenate()([x, s2])
    x = layers.Conv2D(128, 3, activation='relu', padding='same')(x)

    x = layers.Conv2DTranspose(64, 3, strides=2, padding='same')(x)
    x = layers.Concatenate()([x, s1])
    x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)

    x = layers.Conv2DTranspose(32, 3, strides=2, padding='same')(x)
    x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
    x = layers.Conv2DTranspose(16, 3, strides=2, padding='same')(x)
    x = layers.Conv2D(16, 3, activation='relu', padding='same')(x)

```

```

        outputs = layers.Conv2D(1, 1, activation='sigmoid', dtype='float32')
        return models.Model(inputs=backbone.input, outputs=outputs)

def bce_dice_loss(y_true, y_pred):
    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    dice = dice_loss(y_true, y_pred)
    return .1*bce + dice

def process_binary_seg(image_path, mask_path, is_training=True):
    img = tf.io.read_file(image_path)
    img = tf.image.decode_png(img, channels=3)
    img = tf.image.resize(img, (224, 224))
    img = tf.cast(img, tf.float32)

    mask = tf.io.read_file(mask_path)
    mask = tf.image.decode_png(mask, channels=3)
    mask = tf.image.resize(mask, (224, 224), method='nearest')
    mask = tf.cast(mask, tf.float32)

    if is_training:
        combined = tf.concat([img, mask], axis=-1)
        combined = tf.image.random_flip_left_right(combined)
        img, mask = tf.split(combined, [3, 3], axis=-1)

    red_channel = mask[..., 0]
    mask_bin = tf.cast(red_channel > 50, tf.float32)
    mask_bin = tf.expand_dims(mask_bin, axis=-1)

    return img, mask_bin

def create_binary_seg_dataset(paths, masks, is_training=True):
    ds = tf.data.Dataset.from_tensor_slices((paths, masks))
    if is_training: ds = ds.shuffle(buffer_size=1000)

    ds = ds.map(lambda x, y: process_binary_seg(x, y, is_training=is_training))

    ds = ds.batch(BATCH_SIZE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

train_seg_ds = create_binary_seg_dataset(train_paths, train_masks, is_training=True)
val_seg_ds = create_binary_seg_dataset(val_paths, val_masks, is_training=False)
test_seg_ds = create_binary_seg_dataset(test_paths, test_masks, is_training=False)

```

✓ Classifier Demo

Start coding or [generate](#) with AI.

```
# Load saved model
model = tf.keras.models.load_model('/content/drive/MyDrive/AI-ML-Heal-

# Run inference on the testing dataset
test_images, test_labels = next(iter(test_ds))
predictions = model.predict(test_images)

mapping = {
    0: 'Negative',
    1: 'Suspicious',
    2: 'Positive'
}

plt.figure(figsize=(15, 5))

# Plot 3 images & model predictions
for _, i in enumerate(range(6, 9)):
    ax = plt.subplot(1, 3, _ + 1)
    img = test_images[i].numpy()
    img = img/255
    plt.imshow(img)

    true_class = np.argmax(test_labels[i])
    pred_class = np.argmax(predictions[i])
    confidence = np.max(predictions[i]) * 100

    title = f"True: {mapping[true_class]} | Prediction: {mapping[pred_
    plt.title(title)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

1/1 ————— 5s 5s/step

True: Negative | Prediction: Negative
Confidence: 79.2%



True: Negative | Prediction: Negative
Confidence: 60.0%



True: Positive | Prediction: Positive
Confidence: 62.6%





✓ Segmentation Demo

```
custom_objs = {
    'dice_loss': dice_loss,
    'dice_coef': dice_coef,
    'bce_dice_loss': bce_dice_loss
}

# Load saved model
unet_model = tf.keras.models.load_model('/content/drive/MyDrive/AI-ML-

samples = []
for images, masks in test_seg_ds:
    # Run model on 5 images from testing set
    preds = unet_model.predict(images, verbose=0)
    for i in range(len(images)):
        if np.sum(masks[i]) > 0:
            samples.append((images[i], masks[i], preds[i]))
        if len(samples) >= 5: break
    if len(samples) >= 5: break
```

```
plt.figure(figsize=(10, 15)) # 5 rows

for i, (img, mask, pred) in enumerate(samples):
    img_disp = img.numpy().astype("uint8")
    true_mask = mask.numpy().squeeze()
    pred_mask = (pred > 0.5).astype("uint8").squeeze()

    # Column 1: Image
    plt.subplot(5, 3, i*3 + 1)
    plt.imshow(img_disp)
    plt.axis("off")
    if i == 0: plt.title("Input Image")

    # Column 2: Ground Truth
    plt.subplot(5, 3, i*3 + 2)
    plt.imshow(true_mask, cmap='gray')
    plt.axis("off")
    if i == 0: plt.title("Ground Truth")

    # Column 3: Prediction
    plt.subplot(5, 3, i*3 + 3)
    plt.imshow(pred_mask, cmap='gray')
    plt.axis("off")
    if i == 0: plt.title("Prediction")

plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:
saveable.load_own_variables(weights_store.get(inner_path))
```

Input Image



Ground Truth



Prediction





